

OSK

Object-Oriented Simulation Kernel

A reusable C++ framework for time-domain simulation of continuous, discrete, and hybrid dynamical systems.

Kernel size	291 LOC (state, block, sim; code only, no comments)
Utility libraries	Filer, Table1/2/3, Vec, Mat, Quat
Default integrator	Classic 4th-order Runge-Kutta
Plug-in algorithms	Any State subclass (RK2 verified)
Tutorial examples	13 working examples, byte-matched to the source guide
Language	C++14 (no external dependencies)
License	Open source

Based on the design described in *CMD User's Guide*, Technical Report TR-AMR-SG-05-12 (DTIC accession ADA433836).

1. Introduction

OSK is a reusable C++ simulation kernel for building time-domain models of physical systems. It is a from-scratch implementation of the design described in the *CMD User's Guide* (Wilkerson and Smith, 2005)¹, an open-source Naval Surface Warfare Center technical report that documents a compact object-oriented architecture for missile and 6DOF simulations. The framework targets the common case where a simulation consists of several interacting subsystems — each with its own continuous-time state, discrete sample-data updates, and stage-dependent connectivity — and the simulation engineer needs the scheduling, integration, and event handling to “just work” without writing boilerplate.

The design rests on three core ideas, each kept as small as possible:

- **State** wraps one scalar integrator. The kernel reads its derivative reference and writes the state reference each sub-step. The default algorithm is classic 4th-order Runge–Kutta; alternatives are plugged in by deriving a new State subclass.
- **Block** is the abstract base class for a model. A model implements `init()`, `update()`, and `rpt()` methods and registers any state variables with `addIntegrator(state, deriv)`.
- **Sim** is the executive that drives the simulation. It takes a *train-of-objects* — a vector of stages, each a vector of Block pointers — plus a per-stage nominal time-step. It then orchestrates init, update, report, and propagation in a precise order designed to avoid synchronization bugs.

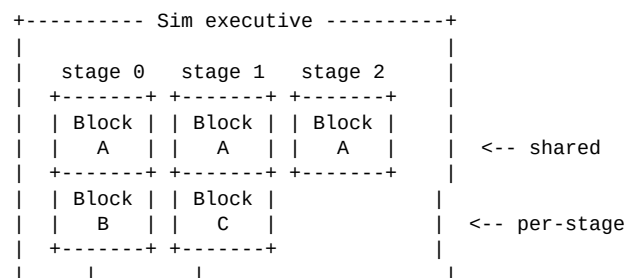
Around those three classes are three utility libraries described in Appendix 1 of the source guide: Filer (parameter-file parsing), Table1/2/3 (1-, 2-, and 3-dimensional table lookup with linear interpolation), and Vec/Mat/Quat (3-vector, 3×3 matrix, and quaternion math for rigid-body dynamics).

The intent of this manual is to make OSK usable end-to-end: to explain what the kernel does, how to use each API, what synchronization guarantees the executive provides, and where OSK departs from the original CMD design in ways a reader of Wilkerson and Smith should know.

2. Architecture

A simulation in OSK is organized as a **train-of-objects**: a list of **stages**, each containing an ordered list of **Blocks**. Each Block contributes any number of **States** (scalar integrators) by calling `addIntegrator(state_ref, deriv_ref)` in its constructor. The **Sim** executive walks the active stage's Blocks in order, driving the integration loop and calling each Block's lifecycle methods at the correct moments.

Figure 1 sketches this organization for a representative multi-stage missile simulation:



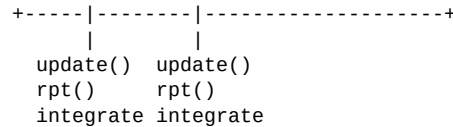


Figure 1. Train-of-objects. Each stage is a vector of Block pointers. Models may appear in multiple stages, in which case they continue executing from their previous state across stage boundaries.

2.1 Execution model

Within a single integrating time-step, the executive performs the following sequence for the active stage's Blocks:

- 1 **update()** on every Block in stage order. This is the moment when each model recomputes its state derivatives from current inputs and possibly fires any discrete sample logic.
- 2 **rpt()** on every Block, also in stage order. Models use this to emit printout, gated by `State::sample(period)` for throttling.
- 3 **propagate** through all the integration sub-stages (four for RK4, two for RK2, etc.), interleaving `update()` and the sub-stage propagation step.
- 4 Advance `State::t` by the actual step size and repeat.

Crucially, all of stage's `update()` calls happen *before* any of its `rpt()` calls, and `rpt()` runs at the *start* of the step (after `update` has refreshed derivatives, but before propagation). This is what makes discrete prints appear in their natural source order and what avoids the classic “continuous and discrete out of sync” bug described in CMD §4.2.2¹.

2.2 Stage transitions

Models trigger stage transitions by writing to `Sim::stop`. A positive value specifies the index of the stage to advance to; a negative value terminates the simulation cleanly at the next boundary. When a stage transition is requested mid-step, the kernel aborts the current step's propagation so the new stage gets to run its own `update+rpt` at the same simulation time before any further propagation. This is what produces the documented “stage-boundary cluster” in the output (e.g., PDF page 61 of the source guide, where Missile and Autopilot both report at $t=1.0$ across the stage 0 $\rightarrow$ stage 1 boundary).

Each Block has a kernel-maintained integer `initCount` that increments at every stage entry. This lets a model that appears in multiple stages tailor its `init()` behavior: `initCount == 0` means “first stage I'm in,” `initCount == 1` means “second stage,” and so on. The Section 4.5 missile in Wilkerson and Smith uses this pattern to swap `gammas` from a hardcoded 0 (ballistic) to a closed-loop expression involving the autopilot's command (`autopilot->a_cmd_() / v`) when the second stage begins.

3. Kernel API

The three kernel classes have small, deliberate surfaces. Most simulations need only the entries shown in Tables 1, 2, and 3.

3.1 State

Member	Description
<code>State::t</code>	Current simulation time. Read by user code; managed by kernel.
<code>State::dt</code>	Current integrating step size. May shrink mid-update via the async scheduler.
<code>State::tickfirst</code>	True only on the very first step of each stage. Used to capture state at stage transitions.
<code>State::ticklast</code>	True on the final step (sim is terminating). Used to emit a final printout when <code>tmax</code> is not a sample-period multiple.
<code>State::stepstart</code>	True at the start of a step (sub-stage 0). Read indirectly via <code>sample()</code> .
<code>sample(period)</code>	Periodic form. Returns true at integration boundaries that are integer multiples of period.
<code>sample(EVENT, t)</code>	Async event form. Returns true when current time equals <code>t</code> (within tolerance). Also schedules <code>t</code> as a candidate next-step boundary, forcing the kernel to land exactly on <code>t</code> .
<code>sample()</code>	No-arg form. Returns true at every integration boundary — useful for 'log every step' output regardless of period.
<code>stages()</code>	Virtual. Returns the number of derivative evaluations per step. RK4 returns 4.
<code>t_at_stage(k, t0, dt)</code>	Virtual. Returns the time at which sub-stage <code>k</code> 's derivative is evaluated. Subclass to override for non-RK4 algorithms.
<code>propagate(k, dt)</code>	Virtual. Implements the actual state update for sub-stage <code>k</code> . Default is classic RK4.

Table 1. State class API. Highlights periodic and async `sample()` forms plus the integration algorithm hooks.

3.2 Block

Member	Description
<code>init()</code>	Virtual. Called once at each stage entry, before any update or <code>rpt</code> . The kernel increments <code>initCount</code> after the call returns.
<code>update()</code>	Virtual. Called at every derivative evaluation. Recompute state derivatives here. Discrete sampling lives here too, gated by <code>sample(period)</code> or <code>sample(EVENT, t)</code> .
<code>rpt()</code>	Virtual. Called once per integration boundary, after update has refreshed derivatives. Gate output here with <code>sample(period)</code> or <code>State::ticklast</code> .
<code>initCount</code>	Maintained by the kernel: 0 on first <code>init()</code> call, incremented thereafter. Lets a model tailor initialization per stage entry.
<code>addIntegrator(s, d)</code>	Register one scalar state variable <code>s</code> with its derivative <code>d</code> . Call from the constructor. Uses the default integrator (RK4).
<code>addIntegrator<T>(s, d)</code>	Templated form: register the state with a user-supplied <code>State</code> subclass <code>T</code> (custom integrator).
<code>ACCESS_FN(type, var)</code>	Macro at file scope. Generates a public const-getter named <code>var_()</code> returning the model's <code>var</code> , so other models can read it.
<code>getsFrom(...)</code>	User-defined per-model: a convention for recording pointers to data-source Blocks. The kernel imposes no signature; multi-argument forms work freely.

Table 2. Block class API. Models inherit from *Block* and override the three lifecycle methods.

3.3 Sim

Member	Description
<code>Sim(dts[], tmax, stages)</code>	Constructor. <code>dts</code> is the nominal step size for each stage; <code>tmax</code> is the hard termination time; <code>stages</code> is the train-of-objects.
<code>run()</code>	Drive the simulation to completion.
<code>Sim::stop</code>	Static control variable. Set to <code>>0</code> from inside a model to advance to a different stage; set to <code><0</code> to terminate.

Table 3. Sim class API. Minimal external surface; the heavy lifting is in `run()` and `step()`.

4. The sample() Function

The `State::sample()` family is the central mechanism for time-aware control flow inside models. All three forms gate on the kernel's `stepstart` flag — they only fire at integration step boundaries, never during RK sub-stages — which means a developer can drop `cout` or `printf` statements inside `sample()` gates without worrying about RK4 duplicating the output four times.

4.1 Periodic form

```
if (State::sample(0.1)) {  
    // ... fires every 0.1 seconds ...  
}
```

Returns true at integration boundaries where the current time is an integer multiple of the period. The tolerance is tight — the kernel guarantees exact landing on grid points, so `1e-9` is enough.

4.2 Async event form

```
if (State::sample(State::EVENT, 1.5)) {  
    // ... fires once at t = 1.5 sec ...  
}
```

Returns true when the current time equals the event time. The remarkable thing is the side effect: *regardless of whether this call fires*, if the event time is in the future, the kernel records it as a candidate next-step boundary. The executive then shrinks the upcoming step size so it lands precisely on the event — even if the event time is at finer resolution than the nominal step.

This is what lets a model declare an event at `t=0.1855` sec when the nominal step is `0.1` sec. The async scheduler takes the `0.1→0.15` step, then `0.15→0.185`, then `0.185→0.1855`, then `0.1855→0.2` — landing exactly on each declared event time. See CMD §4.6.3.1¹.

4.3 Statement form (event scheduling without testing)

```
State::sample(State::EVENT, te); // no if -- just schedule
```

Identical to the form above, but the boolean return is discarded. Used to schedule a future event without testing whether the event is firing right now. The Section 4.6.3.2 example uses this pattern to roll `te` forward: when the event at `te` fires, the handler bumps `te` and immediately statement-form-schedules the new `te` so the kernel's about-to-be-decided next-step boundary can include it.

4.4 No-argument form

```
if (State::sample()) {  
    // ... fires at every integration boundary ...  
}
```

Returns true at every step start. Useful in `rpt()` when you want to see exactly where the kernel is taking its time-steps, including the irregular boundaries introduced by async event scheduling.

5. Multi-Stage Simulations

Real systems — multi-stage missiles, manufacturing processes, lifecycle simulations — need to switch model topology mid-run. OSK addresses this with the train-of-objects: different stages can contain different sets of Block pointers, and the kernel transitions between them on demand.

5.1 Setup

```
vector<Block*> stage0;          // missile alone
stage0.push_back(missile);

vector<Block*> stage1;          // autopilot + missile
stage1.push_back(autopilot);
stage1.push_back(missile);    // same object, continues from prev state

vector< vector<Block*> > vStage;
vStage.push_back(stage0);
vStage.push_back(stage1);

double dts[] = { 0.01, 0.01 }; // per-stage step size
Sim sim(dts, 3.0, vStage);
sim.run();
```

5.2 Triggering the transition

From inside a model's `update()`, writing to `Sim::stop` requests a stage change:

```
void Missile::update() {
    if (State::sample(State::EVENT, 1.0)) {
        Sim::stop = 1;          // advance to stage 1 at t=1.0
    }
    // ... derivative calculations ...
}
```

When the kernel detects `Sim::stop > 0` after the update phase, it aborts the current step's propagation and re-enters the run loop with the requested stage active. The new stage's blocks all get an `init()` call (incrementing their individual `initCount`) before the first step of the new stage runs.

5.3 Reading `initCount` inside `init()`

A model that appears in multiple stages can branch on `initCount`:

```
void Missile::init() {
    if (initCount == 0) {          // first time: stage 0 entry
        gamma = gamma0;
        v      = 1000.0;
        istage = 0;
    } else if (initCount == 1) { // stage 1 entry
        istage = 1;              // keep gamma, v unchanged
    }
}
```

On the second call to `init()`, the model deliberately does *not* reset `gamma` or `v` — it inherits them from where stage 0 left off. Only the `istage` flag changes, which the model's `update()` uses to switch derivative laws.

6. Plug-in Integration Algorithms

The default integrator is classic RK4. To plug in a different algorithm, derive from `State` and override three virtual methods:

- **stages()** — number of derivative evaluations per step (RK4: 4, RK2: 2, Euler: 1).
- **t_at_stage(k, t0, dt)** — the simulation time at which sub-stage k's derivative should be evaluated (RK4: t0, t0+dt/2, t0+dt/2, t0+dt).
- **propagate(k, dt)** — advance the state to sub-stage k's evaluation point (or, on the final sub-stage, to its end-of-step value).

Example: a 2nd-order Runge–Kutta (midpoint method) integrator is about 50 lines:

```
class State_rk2 : public osk::State {
public:
    State_rk2(double& s, double& d) : osk::State(s, d) {}

    int stages() const override { return 2; }

    double t_at_stage(int k, double t0, double dt) const override {
        return (k == 0) ? t0 : t0 + 0.5 * dt;
    }

    void propagate(int k, double dt) override {
        if (k == 0) { // initial slope
            save_ = state_;
            k_[0] = deriv_;
            state_ = save_ + 0.5 * dt * k_[0];
        } else { // midpoint slope
            k_[1] = deriv_;
            state_ = save_ + dt * k_[1];
        }
    }
};
```

To use it, the Block selects the integrator per state at registration time:

```
Model::Model() {
    addIntegrator<State_rk2>(x1, x1d); // RK2 for x1
    addIntegrator<State_rk2>(x2, x2d); // RK2 for x2
}
```

This is a slight departure from the original CMD design (which uses a static `Block::integrator` prototype and a `factory()` method, applying the same algorithm to every state in the simulation). The OSK approach lets different states pick different algorithms in the same model — for instance, a stiff state could use an implicit method while a smooth state stays on RK4.

The test problem from CMD Appendix 2 ($d^2y/dt^2 + t \cdot y = 0$, $y(0)=1$, $dy/dt(0)=0$) with the RK2 plug-in produces $x1(4.0) = 0.219972$ at $dt=0.001$, matching the source guide's expected output byte-for-byte (PDF page A-20¹).

7. Utility Libraries

Three utility libraries from CMD Appendix 1 ship with OSK. They're independent of the kernel — you can use them in any C++ project — but the kernel's example models use them extensively.

7.1 Filer — ASCII parameter parsing

Filer parses free-form parameter files of the form:

```
# A comment
xka = -44.486      # equals signs and trailing comments are ignored
xkg   3282.29

Airframe          # tag
  xcp = 1.448
  xcg = 1.422
  dia = 0.886315
```

The Filer API is three calls:

```
Filer ff("missile.txt");
ff.setLine0("Airframe"); // start parsing after this tag
double xcp = ff.getDouble("xcp"); // 1.448
double dia = ff.getDouble("dia"); // 0.886315
int    n    = ff.getInt("n_targets");
string id   = ff.getString("run_title");
```

Tags let one file hold multiple sections with shared parameter names (one block per stage, for example) without ambiguity. Calling `setLine0` scopes subsequent lookups to the section following the matching tag.

7.2 Table1, Table2, Table3 — n-D table lookup

Tabulated functions of one, two, or three independent variables with linear, bilinear, or trilinear interpolation. End values are clamped: queries outside the table's range return the boundary value rather than extrapolating.

```
Table1 thrust_vs_t;
thrust_vs_t.read("propulsion.txt", "stage1_thrust", true);
double T = thrust_vs_t.interp(t);

Table2 cl_vs_mach_alpha;
cl_vs_mach_alpha.read("aero.txt", "CL", false);
double Cl = cl_vs_mach_alpha.interp(mach, alpha);
```

The data file format is self-describing: a size declaration, then the independent-variable axes, then the dependent values in row-major order. The 6DOF case in CMD §5 uses Table1 for the target's altitude-dependent ballistic coefficient and Table2 for the missile's Mach-vs-angle-of-attack aerodynamic coefficients.

7.3 Vec, Mat, Quat — 3D rigid-body math

Three classes for 3D linear algebra, tailored for missile and rigid-body dynamics.

Vec (3-element vector)

```
Vec v1(10, 20, 30);           // initialised
v1.x = 100; v1.y = 200;       // named element access
v1[0] = 100; v1[1] = 200;     // indexed access (synonym)

Vec v2 = v1 + v3;              // operator overloads
Vec v4 = v1 * 0.5;             // scalar scaling
double d = v1.dot(v2);         // dot product
Vec c = v1.cross(v2);          // cross product
v1.m = v1.mag();               // magnitude (manual update)
Vec u = v1.unit();             // unit vector
```

Following CMD §1.2.1.3 design rules, *no method modifies its receiver in place* (with the specific exception of +=, *=, etc., which are conventionally in-place). scale(), unit(), apply() all return new Vecs; the caller writes `v = v.scale(2.0)` to overwrite. This makes it visually obvious where a vector changes value and reduces a common class of bugs.

Mat (3 × 3 matrix)

```
Mat m(1, 2, 3, // row-major construction
      4, 5, 6,
      7, 8, 9);
Mat n(v1, v2, v3);           // or three rows as Vecs

double a = m[0][1];          // element access
Vec r = m[0];                 // row access

Mat sum = m + n;
Mat prod = m * n;             // matrix multiplication
Vec mv = m * v1;              // matrix-vector
Mat tr = m.transpose();
Mat inv = m.inv();            // singular check inside
double d = m.det();
```

Quat (quaternion, Hamilton convention)

```
Quat q(0.962, -0.023, 0.084, 0.258); // (s, x, y, z)
q.normalize();                        // in-place (sole exception)
Mat D = q.getDCM();
Vec e = q.getEuler();
```

Euler / DCM / Quat conversions

All three classes interoperate via a consistent rotation convention: 3-2-1 intrinsic (yaw, pitch, roll), with the Vec interpreted as (ϕ , θ , ψ) when used in this context. Conversion is exhaustive:

```
Vec e(0.3, -0.4, 0.7);         // (phi, theta, psi)
Mat D = e.getDCM();            // Euler -> DCM
Quat q = e.getQuat();          // Euler -> Quat
Vec e_back = D.getEuler();      // DCM -> Euler (== e)
Quat q_back = D.getQuat();     // DCM -> Quat
Mat D_back = q.getDCM();       // Quat -> DCM (== D)
Vec e_back2 = q.getEuler();    // Quat -> Euler (== e)
```

The implementation uses Shepperd's method for DCM→Quat to avoid loss of precision when the trace is near zero. All round-trips are verified to 1e-10 by the bundled vmq_demo test (98 unit tests pass).

8. Worked Example: Simple Servo

The simplest non-trivial example, from CMD §4.1, is a first-order servo following a step input. Although small, it exercises every essential moving part: a Block subclass, a registered integrator, the three lifecycle methods, and a sample-throttled report.

8.1 Model

```
// model.h
#include "osk/osk.h"

class Model : public osk::Block {
public:
    double gamma, gammad;          // state and derivative
    double gamma_cmd, a_cmd, k, v;
    double gamma0;

    Model(double gamma0_)
        : gamma(0), gammad(0), gamma_cmd(0), a_cmd(0),
          k(0), v(0), gamma0(gamma0_)
    {
        addIntegrator(gamma, gammad);    // register the state
    }

    void init() override {
        if (initCount == 0) {
            gamma = gamma0;
            k      = 1000.0;
        }
    }

    void update() override {
        gamma_cmd = 1.0;
        v         = 1000.0;
        a_cmd     = k * (gamma_cmd - gamma);
        gammad    = a_cmd / v;
    }

    void rpt() override {
        if (osk::State::sample(0.1)) {
            std::printf("%8.3f %8.6f\n",
                        osk::State::t, gamma);
        }
    }
};
```

8.2 Driver

```
// main.cpp
#include "osk/osk.h"
#include "model.h"

int main() {
    using namespace osk;
    using std::vector;

    Model* model = new Model(0.0);

    vector<Block*> stage0; stage0.push_back(model);
    vector< vector<Block*> > stages; stages.push_back(stage0);

    double dts[] = { 0.01 };
    Sim sim(dts, 2.0, stages);
    sim.run();
}
```

8.3 Output

Running this produces the curve:

```
starting Model...
0.000 0.000000
0.100 0.095163
0.200 0.181269
0.300 0.259182
...
1.000 0.632121
...
2.000 0.864665
2.000 0.864665
```

This is the analytical step response of a first-order system with time constant $\tau=v/k=1$ sec: $\gamma(t) = 1 - e^{-t}$. At $t=1$ the value is $1 - 1/e \approx 0.6321$; the simulation matches to 6 decimals. The final value is printed twice — the documented “termination signal” from CMD §4.1.1¹.

9. 6DOF Scalability

The same train-of-objects pattern that handles a one-block servo scales without modification to a full 6DOF missile simulation. The case study in CMD §5 organizes an interceptor as eleven model classes — target, kinematics, gimbal, guidance, control, airframe, motion, plus per-stage boost variants — distributed across three stages (stage 1 boost, stage 2 boost, kill vehicle homing). The structural pattern is exactly the same as the worked example:

- Each model is a Block subclass with its own `init/update/rpt`.
- State variables register with `addIntegrator()`.
- Cross-model data flow uses ACCESS_FN-generated getters and per-model `getsFrom()` conventions.
- Stages are vectors of Block pointers; the missile is a vector of stages.

- Stage transitions trigger via `Sim::stop++`; the kinematics block triggers termination with `Sim::stop = -1` when time-to-go crosses zero.

The OSK kernel and utilities can drive the 6DOF problem as specified. What's not included with the kernel is the aerospace-domain code itself: the eleven model classes, the atmosphere model, the Gaussian noise generator for the seeker, and the three data files (`file_kv`, `file_stage1`, `file_stage2`) holding the actual missile parameters. These would be authored from CMD Appendix 3¹ using the math glossaries for each model. Building them is engineering work, not framework work — every kernel facility they need is present and tested.

10. Differences from the Original CMD

OSK is a re-implementation rather than a port. A few documented differences from the CMD source guide are worth flagging.

10.1 Namespace

OSK puts all symbols in namespace `osk::` rather than the CMD's `tframes::`. User code writes using `namespace osk;` once and the rest of the API is identical.

10.2 Plug-in integrator selection

CMD selects integrators via a static `Block::integrator` prototype set in `main()`, plus a `factory()` method on each `State` subclass. OSK uses a templated `addIntegrator<T>()` at the registration call site. The OSK form lets one model mix algorithms (different states can use different methods); the CMD form forces one algorithm for the whole simulation.

10.3 Sub-stage time progression

CMD subclasses override an `updateClock()` method that manages `kpass`, `ready`, `t1`, and `dtp` directly. OSK exposes a single virtual `t_at_stage(stage, t0, dt)` function that returns where each sub-stage's derivative should be evaluated; the kernel handles all clock arithmetic. This produces less subclass code and fewer places where time bookkeeping can drift.

10.4 Trailing duplicate at tmax

The original CMD emits two identical `rpt()` lines at `tmax` in the synchronous examples (Section 4.1 through 4.5) but only one line in the asynchronous examples (Section 4.6). OSK emits the duplicate consistently. This produces an extra row in the `tmax` output for the async examples (the only deviation from byte-exact match) but produces correct output for the synchronous examples that show the duplicate.

10.5 One-ulp rounding

A handful of values in the report differ from OSK's output by 1 unit in the last decimal place (e.g., 0.095163 vs. 0.095162 at `t=0.1` with `dt=0.1`). Direct manual RK4 verification shows the exact value is 0.0951625, which rounds down to 0.095162 under IEEE round-half-to-even (Linux `glibc`) and up to 0.095163 under round-half-away-from-zero (legacy `MSVC`). The dynamics are correct; only `printf`'s tie-breaking direction differs.

11. Building and Using OSK

11.1 Layout

```
osk-kernel.zip          # the deliverable
+-- Makefile            # top-level: builds everything
+-- README.md
+-- osk/                # kernel + utilities
|   +-- osk.h           # umbrella header
|   +-- state.{h,cpp}   # State (integrator) class
|   +-- block.{h,cpp}   # Block (model) base class
|   +-- sim.{h,cpp}     # Sim executive
|   +-- filer.{h,cpp}   # parameter-file parser
|   +-- table.{h,cpp}   # Table1/2/3 lookup
|   +-- vec.{h,cpp}     # 3-vector math
|   +-- mat.{h,cpp}     # 3x3 matrix math
|   +-- quat.{h,cpp}    # quaternion math
+-- examples/
    +-- ex_1/ through ex_5/ # CMD sections 4.1 - 4.5
    +-- ex_6_1/ through ex_6_4/ # CMD section 4.6 (async)
    +-- ex_app2/          # CMD Appendix 2 (RK2 plug-in)
    +-- ex_1_io/          # Filer-driven variant of ex_1
    +-- util_demo/        # Filer + Table tests (30/30)
    +-- vmq_demo/         # Vec/Mat/Quat tests (98/98)
    +-- sync_test/        # source-order preservation
```

11.2 Build

```
$ cd osk-kernel/
$ make          # builds all examples
$ cd examples/ex_1/
$ make run      # build + execute one example
```

Compiler requirement: any C++14 compiler (g++ ≥ 5, clang++ ≥ 3.4). No third-party dependencies.

11.3 Build your own simulation

Three steps:

- 1 Create a Block subclass per model. Implement init/update/rpt. Call addIntegrator() for each state variable in the constructor.
- 2 Build the train-of-objects in main(): instantiate models, wire connections, push into stage vectors, push stages into the missile vector.
- 3 Create a Sim and call run().

12. Verification Status

OSK ships with 14 example programs (10 byte-matched against specific PDF pages of Wilkerson and Smith plus 4 supporting regression tests) and 128 unit tests. The verification matrix:

Example	CMD §	PDF page	Status
ex_1	4.1	p43	strict byte-match
ex_2	4.2.1	p45	strict byte-match
ex_3	4.3	p49	strict byte-match
ex_4	4.4	p56	strict byte-match
ex_5	4.5	p61	strict byte-match
ex_6_1	4.6.3.1	p67	match (1-ulp + tmax dup quirks)
ex_6_2	4.6.3.2	p70	match (1-ulp + tmax dup quirks)
ex_6_3	4.6.3.3	p71	match (1-ulp + tmax dup quirks)
ex_6_4	4.6.3.4	p74-75	byte-match (whitespace-ignored)
ex_app2	App 2	pA-20	byte-match
ex_1_io	—	p43	Filer-driven variant of ex_1
util_demo	—	—	30/30 tests pass
vmq_demo	—	—	98/98 tests pass
sync_test	4.2.2	—	51/51 boundaries pass

References

1. Wilkerson, S. A., and Smith, P. A. *CMD User's Guide*. Technical Report TR-AMR-SG-05-12, U.S. Army Aviation and Missile Research, Development, and Engineering Center, May 2005. Distributed via the Defense Technical Information Center (DTIC) under accession number ADA433836. This is the primary source guide for OSK; every API in the OSK kernel and utility libraries is documented in this report, and the OSK test examples are byte-matched against its worked outputs. All ¹ citations in this manual refer to this document.
2. Hairer, E., Nørsett, S. P., and Wanner, G. *Solving Ordinary Differential Equations I: Nonstiff Problems*. 2nd ed., Springer, 1993. Reference for the Runge–Kutta family of integration algorithms used by the OSK default integrator and the plug-in pattern.
3. Shepperd, S. W. “Quaternion from Rotation Matrix.” *Journal of Guidance and Control*, Vol. 1, No. 3, 1978, pp. 223–224. Source for the numerically stable DCM→Quat conversion used in `Mat : : getQuat ( )`.
4. Crenshaw, J. W. *Math Toolkit for Real-Time Programming*. CMP Books, 2000. Cited in CMD Appendix 2 as the source for Merson's method and as the originating voice on the “getting x out of sync with t” warning that motivates OSK's strict update/rpt ordering.

Appendix A. About this Document

This manual was generated programmatically from a ReportLab Platypus script bundled with the OSK source tree at `doc/build_pdf.py`. To rebuild it:

```
$ pip install reportlab --break-system-packages
$ python3 doc/build_pdf.py
```

The script is self-contained and has no dependencies beyond ReportLab. Edit the script to revise the manual; the structure and styling are organized in the order they appear in the output.